# Software Engineering




Dr.ABDELKOUI Feriel

Abdelhamid Mehri University Constantine 2.NTIC

feriel.abdelkoui@univ-constantine2.dz

Course for 03rd year. computer security for Engineering

5.0

janvier 2025

*Paternité : http://creativecommons.org/licenses/by/4.0/fr/*

# Table des matières

**Objectifs** **3**

**I - chapitre 03 uml** **4**

1. Objectives ..... 4
2. definition ..... 4
3. why model ..... 4
4. The Object-Oriented Approach: ..... 5
5. The Class: ..... 5
6. Relations between classes ..... 6
7. History ..... 6
8. UML ..... 6
9. UML diagrams ..... 7

**Bibliographie** **8**

**Webographie** **9**

# Objectifs

By the end of this course, you will be able to:

- Define the principles, objectives, and expected qualities of software engineering.
- Describe the different phases and models of the software development life cycle
- Compare various system design methods, both static and dynamic.
- Apply appropriate design methodologies in a system development context
- Use UML as a standard language for object-modeling.
- Evaluate the effectiveness of different modeling approaches based on project requirements.

# I chapitre 03 uml

Modeling: The action of designing a model in a dedicated modeling language. Modeling involves identifying the interesting or relevant characteristics of a system to study it from the perspective of those characteristics.

## 1. Objectives

*by the end of this chapter you will be able to :*

1. Define the concept of modeling in object-oriented software development.
2. Identify the general elements used in UML notation.
3. Classify UML diagrams based on structural or behavioral views.

## 2. definition

*What is a model?*

- An abstraction that allows for a better understanding of a complex
- object (representing the system with different levels of detail).
- Asimplified representation of a reality (software, building, etc.).
- Modelsare often graphical ("a picture is worth a thousand words").

## 3. why model

- To facilitate the understanding of a system.
- To enable good communication with the client.
- To allow for precise and complete documentation of requirements without knowing the system's details.
- To focus on specific aspects of a system (providing different perspectives).
- To test a multitude of solutions at a lower cost and within reduced timelines.

*Functional Modeling Methods:*

• Functional and systemic methods were first imposed in the 1970s and 1980s. These methods are directly inspired by computer architecture.

• At first, we have a large problem that we break down into several sub problems. We then proceed to decompose the sub-problems into further sub problems. We iterate the process until we obtain simple problems that are easy to understand and therefore easy to program. It is thus a set of precise functions interacting with each other

# 4. The Object-Oriented Approach:

In this approach, the software is considered as a set of objects with specific characteristics that interact with each other.

• The Object:

An object is an autonomous entity , a software entity that represents the basic building

block of object-oriented programming.

An object is characterized by:

• Attributes: Variables representing the data part of the object.

• Methods: Functions or procedures describing its behavior.

# 5. The Class:

Some objects have the same characteristics; they have different identities but:

• A state defined by the same properties (attributes).

• Similar behavior

## *Encapsulation:*

Encapsulation involves hiding the implementation details of an object by defining an interface. The interface specifies the services that are accessible (offered) to users of the object.

• Encapsulation facilitates the evolution of an application because it stabilizes the use of objects: we can modify the implementation of an object's attributes without changing its interface.

## *Inheritance:*

Inheritance is a mechanism for transmitting the properties of a class (its attributes and methods) to a subclass. Inheritance avoids duplication and encourages reuse

## *Polymorphism:*

Polymorphism represents the ability of the same operation to execute differently depending on the context of the class in which it resides. Thus, an operation defined in a super class can execute differently based on the subclass from which it is inherited. Polymorphism increases the genericity of the code.

# 6. Relations between classes

*Aggregation:*

Aggregation is a relationship between two classes, specifying that the objects of one class are components of the other class. A "contains" instances of B.

• The deletion of A does not imply the deletion of B.

• The aggregated element can be shared.

Composition:

The deletion of A leads to the deletion of B.

Association

• Bidirectional semantic connection between classes.

Abstraction of the links that exist between the objects (instances of the associated classes).

• In general, an object cannot act alone; it needs the help of other objects.

Once an object has a link to another object, it can send a message to it to trigger one of its operations.

# 7. History

.

Dominant Methods:

OMT(JamesRumbaugh):Static,dynamic, andfunctional views of a system.

• Originated from the R&D center of General Electric.

• Richandreadable graphical notation.

OOD(GradyBooch):Logicalandphysical views of the system.

• Defined to streamline the development of ADA applications, then C++.

• Doesnotcover the analysis phase in its earlier versions.

• Introduces the concept of a package.

OOSE(IvarJacobson): Covers the entire development cycle.

• Originated from a development center at Ericsson in Sweden.

• Themethodology is based on analyzinguser needs

# 8. UML

*Why UML?*

• UM*L is now a standard defined by the Object Management Group (OMG).

• UML* is primarily used when planning to develop applications with an object-oriented approach (development in Java, C++, etc.).

• Graphical and simple (diagrams)

• Represents entire systems

UML* is not a method; it is an object modeling language.

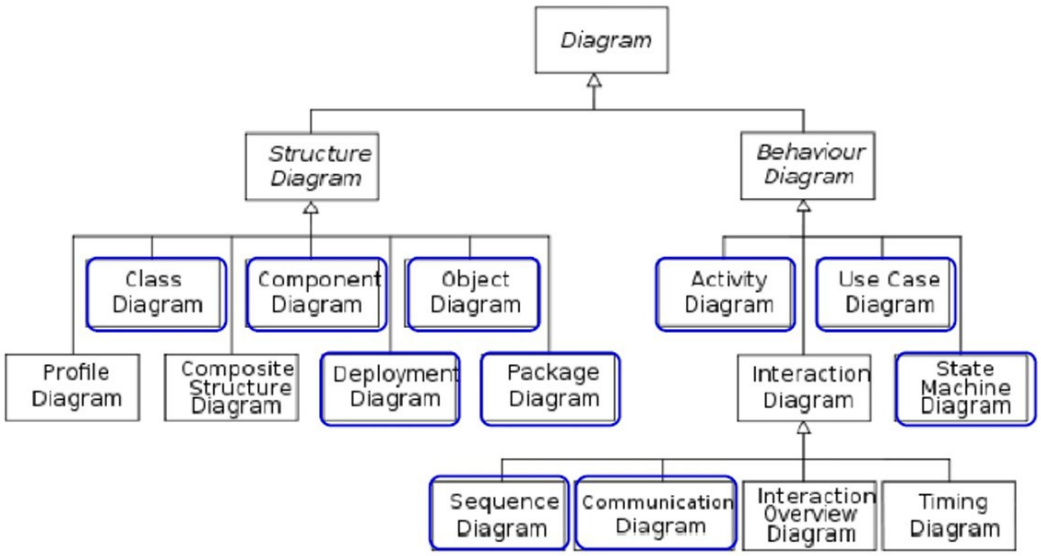
• Iterative and incremental

• Driven by user needs

• Architecture-centric

## 9. UML diagrams

*different diagrams*




.

# Bibliographie

Bibliographies • Uml 2 pratique de la modélisation, Benoît Charroux, Yann Thierry-Mieg, Aomar Osma

Les cahiers du programmeur, Pascal roques

# Webographie

https://fr.slideshare.net/nassimamine3994/uml-2-pratique-de-la-modlisation